Chapter 10
Mining and Consensus

Course Notes

CSE0469 Blockchain: Special Topics in SW Engineering

Yusuf Altunel

Based on Mastering Bitcoin 2e; Andreas M. Antonopoulos; O'Reilly, 2017

Content

- Introduction
- Decentralized Consensus
- Independent Verification of Transactions
- Mining Nodes
- Aggregating Transactions into Blocks
- Constructing the Block Header
- Mining the Block
- Successfully Mining the Block
- Validating a New Block
- Assembling and Selecting Chains of Blocks
- Mining and the Hashing Race
- Consensus Attacks
- Changing the Consensus Rules
- Soft Fork Signaling with Block Version
- Consensus Software Development



Introduction

Bitcoin Money Supply

Bitcoin Economics and Currency Creation

Bitcoin as a Deflationary Money

Introduction



word «Mining»

Evoking the extraction of precious metals

Focus the attention on the reward of mining



Primary purpose

Validation of transactions

Mechanism underpins decentralized clearinghouse



Makes bitcoin special

Decentralized security mechanism

is the basis for P2P digital cash

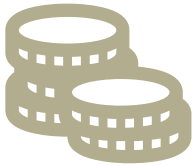
Introduction



Benefits of Mining

Secures the bitcoin system

Enables network wide consensus without central authority



Incentives

Newly minted coins

Transaction fees



Miners

Validate transactions

Record them on the global ledger

Introduction



10 minutes

A new block is mined on 10 minutes on average



«Confirmed» Transactions

become part of a block added to the blockchain

New owners of the bitcoin are allowed to spend



Rewards of miners

New coins created with each new block

Transaction fees from all transactions in the block



Reason of reward

Miners provide security to blockchain

Introduction



Secure transactions

acts as a proof mechanism

to secure the transactions included in the block



Proof-of-Work

The solution to the difficult mathematical problem

Included in the block



Competition

Miners compete to solve a cryptographic hash algorithm

Bitcoin Money Supply



Bitcoin’s money supply

Created by mining process

Similar to central banks issuing new money



Decrease of bitcoin per block

It started at 50 bitcoin per block in January of 2009

halved to 25 bitcoin per block in November of 2012

It halved again to 12.5 bitcoin in July 2016



2140

Decrease is exponential by half every 4 years

All 20.99999998 million bitcoin

will have been issued

approximately the year 2140

Bitcoin Economics and Currency Creation



Minting rate

Bitcoin are “minted”
during the creation of each block
at a **fixed** and
diminishing rate



10 Minutes per Block

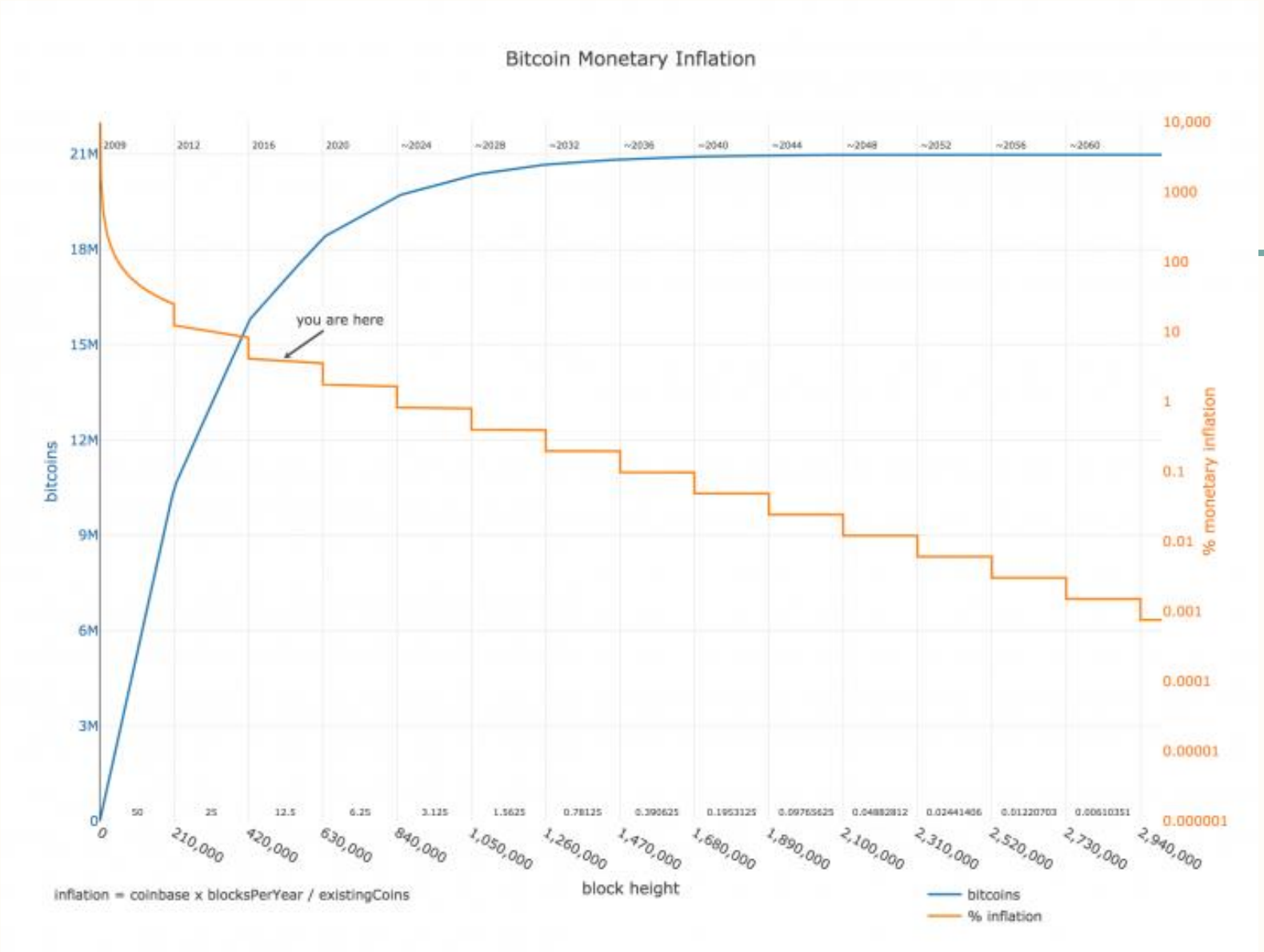
Each block, generated
on average every **10 minutes**
contains entirely new bitcoin
created from nothing.



210 000 blocks

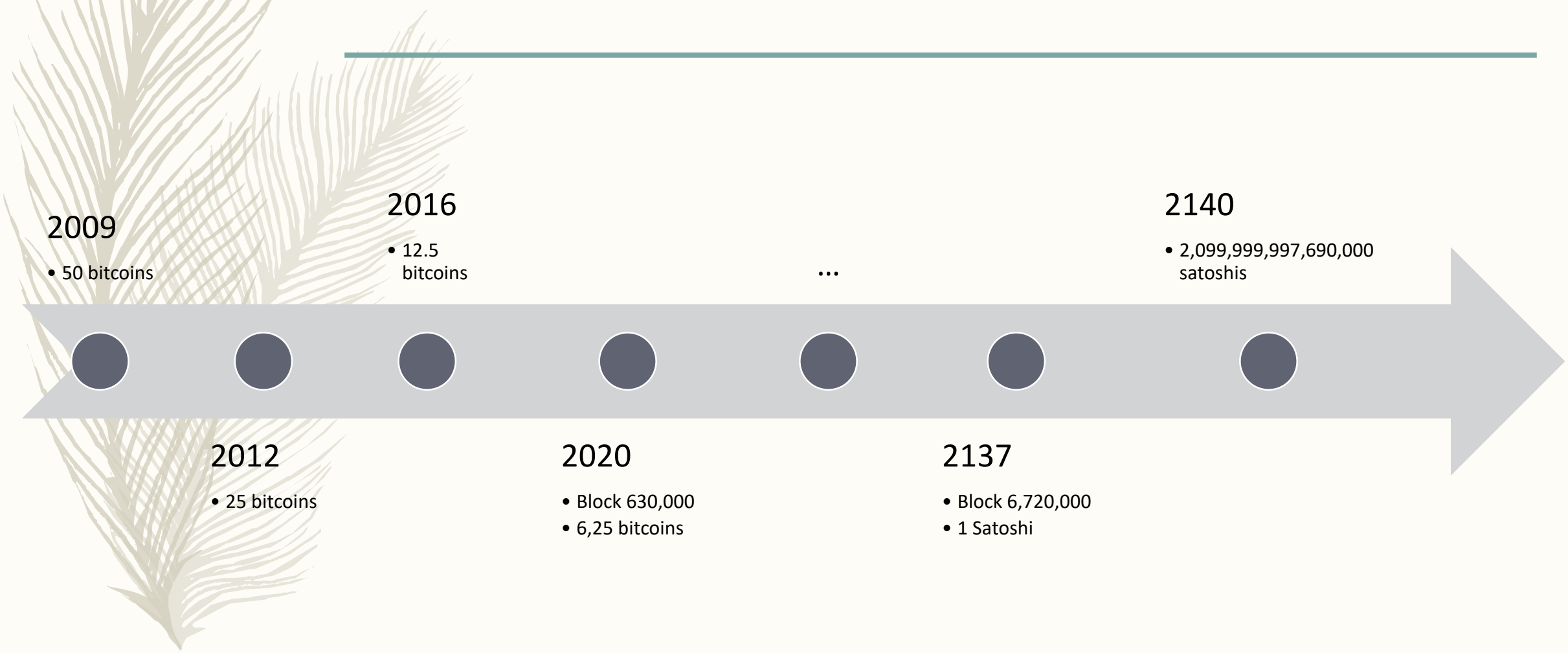
Every 210,000 blocks,
approximately every 4 years
the currency issuance rate
is decreased by 50%.

Bitcoin Money Supply



https://bitcointalk.org/index.php?topic=3414598

Future of Bitcoin Supply



Bitcoin as a Deflationary Money

Bitcoin is deflationary

The opposite of inflation

more purchasing power over time

Consequences of

fixed and diminishing monetary issuance

Supply vs. Demand

produces a mismatch

İn supply and demand

Results in

drives up the value

and exchange rate of a currency

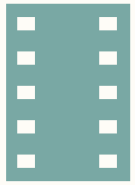
Is Deflation is a Disaster?



Keep instead of spend

people hope prices will fall

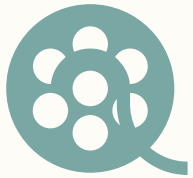
tend to hoard money instead of spending it



“Lost Decade”

Such a phenomenon unfolded

during Japan’s “Lost Decade,”



Deflationary spiral

complete collapse of demand

the currency is pushed into a deflationary spiral.

Deflation in unlimited Money



Inflationary fiat currency

very difficult to enter a deflationary spiral

with the possibility of unlimited printing



Still deflation possible?

if there is a complete collapse in demand

and an unwillingness to print money



Deflation in bitcoin

is not caused by a collapse in demand

but by a predictably constrained supply

The positive aspect of deflation



Hidden taxation

Inflation causes

a slow but inevitable debasement of currency

resulting in a form of hidden taxation



Inflation punishes

savers in order to bail out debtors

including the biggest debtors, governments themselves



Moral hazard

Currencies under government control suffer

from the moral hazard of easy debt issuance

can later be erased through debasement at the expense of savers.

Deflationary Currency



Deflation is a problem

when it is not driven

by rapid economic retraction



Advantage

because the protection from inflation



Reflection

Currency should reflect

the economical conditions



Decentralized Consensus

Trust Model



Clearinghouse

basically verifying and clearing all transactions



Traditional

traditional payment systems

depend on a trust model



Central authority

provide a clearinghouse service

Decentralized Consensus



Bitcoin

has no central authority



Trust to full node

Full node itself as the authority



Exact copy of ledger

every full node has

a complete copy of a public ledger

Decentralized Emergent Consensus



Consensus

Decentralized

Emergent



Decentralized consensus

Different nodes

the same conclusion



Mechanism

every node in the network

arrive at the same conclusion



Emergent consensus

consensus is not achieved explicitly



Election & moment

there is no election

no fixed moment when consensus occurs

Consensus as Emergent Artifact



Asynchronous interaction

thousands of independent nodes,

following simple rules



Emergent artifacts

currency

transactions

payments

the security model



Decentralized

no central authority

No central trust

Processes for Decentralized Consensus



Principles

Independent

Verification



Transactions

each transaction

is verified by **every** full node



Criteria

Verification is based on

a comprehensive list of criteria

Processes for Decentralized Consensus



Proof-of-Work algorithm

Independent aggregation of transactions

into new blocks



Mining

The process is called mining

Performed by mining nodes



Verificaation of blocks

Independent verification

by every node



Independent Verification of Transactions

Independent Verification of Transactions



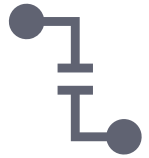
Nodes verify transactions

every bitcoin node that receives a transaction will first verify the transaction



Only valid transactions are propagated

This ensures that only valid transactions are propagated across the network,



Others are discarded

while invalid transactions are discarded at the first node that encounters them.

Verification Checklist

Syntax

- The transaction's syntax and data structure must be correct.

Empty I/O

- Neither lists of inputs or outputs are empty.

Transactions Size

- The transaction size in bytes is less than MAX_BLOCK_SIZE.

Range of Values

- Each output value, as well as the total, must be within the allowed range of values
- less than 21m coins
- more than the *dust* threshold

Input Hashes

- None of the inputs have hash=0, N=–1
- coinbase transactions should not be relayed

Verification Checklist

Time

- nLocktime is equal to INT_MAX,
- or nLocktime and nSequence values are satisfied according to MedianTimePast.

Transactions size

- The transaction size in bytes is greater than or equal to 100.

Signatura operations

- The number of signature operations (SIGOPS) contained in the transaction
- is less than the signature operation limit.

Unlocking Script

- The unlocking script (scriptSig) can only push numbers on the stack,
- and the locking script (scriptPubkey) must match isStandard forms
- this rejects “nonstandard” transactions

Verification Checklist

Matching transaction

- A matching transaction in the pool, or in a block in the main branch, must exist.

Input vs Output

- For each input, if the referenced output exists in any other transaction in the pool,
- the transaction must be rejected.

Referenced output transaction

- For each input, look in the main branch and the transaction pool to find the referenced output transaction.
- If the output transaction is missing for any input, this will be an orphan transaction.
- Add to the orphan transactions pool, if a matching transaction is not already in the pool.

Verification Checklist

Maturity confirmation

- For each input, if the referenced output transaction is a coinbase output
- it must have at least COINBASE_MATURITY (100) confirmations.

Unspent Output Transaction

- For each input, the referenced output must exist and cannot already be spent.

Allowed value ranges

- Using the referenced output transactions to get input values,
- check that each input value, as well as the sum,
- are in the allowed range of values (less than 21m coins, more than 0).

Verification Checklist



Input value<Output values

- Reject if the sum of input values is less than sum of output values.



Too low transaction fee

- Reject if transaction fee would be too low (minRelayTxFee)
- to get into an empty block.



Validating unlocking scriptes

- The unlocking scripts for each input must validate against the corresponding output locking scripts.

Conditions in Bitcoin Core



Bitcoin Core

AcceptToMemoryPool

CheckTransaction

CheckInputs



Evolution

conditions change

over time



Security

to address

new types of denial-of-service attacks



Advancement

to relax the rules

to include more types of transactions

Unconfirmed Transactions: mempool



Mempool

Before propagating

every node builds

a pool of valid but unconfirmed transactions



Names

transaction pool

memory pool

mempool



Function

To verify each transactions

as it is received



Mining Nodes

Miners & Competition

Miners

- Specialized nodes on the bitcoin network
- aggregates transactions into new blocks

Competition

- arrival of a new block has special significance for a mining node
- receiving a valid new block means someone else won the competition and they lost

New round

- the end of one round of a competition
- is also the beginning of the next round



Aggregating Transactions into Blocks

The Coinbase Transaction

Coinbase Reward and Fees

Structure of the Coinbase Transaction

Coinbase Data

Aggregating Transactions into Blocks



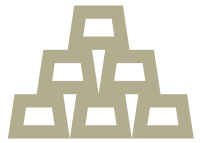
Transaction pool

After validating transactions,
a bitcoin node will add them to the
memory pool, or *transaction pool*,



transactions await

until they can be included (mined)
into a block



Aggregate into a new block

Miners aggregate the transactions
into a *candidate block*.

Construction of new Blocks



10 minutes

Blocks are confirmed and mined in 10 minutes



Preparation of next block

As competing in mining of the current block

The miner also collects incoming valid transactions



Arriving of new block

The miner compares the transactions in new block

With the collected in mempool so far



Remove common transactions

The common transactions are removed from the candidate block



Candidate block

Transactions remain in the memory pool are unconfirmed

and are waiting to be recorded in a new block

Candidate Block



Not a valid block yet

A Block ready to be mined

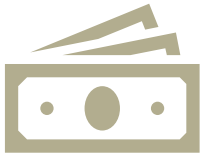
Doesn’t contain Proof-of-Work



Becomes valid

If the miner finds a solution

To the Proof-of-Work algorithm



Candidate block

Contains certain number of valid transactions

With or without transaction fees

Coinbase Transaction



Coinbase

The first transaction in any block

Constructed by miner's node

Contains miner's reward for mining effort

Doesn't consume UTXO inputs



Input

It has only one input called **coinbase**

Creates coin from nothing



Output

It has one output

Payable to miner's own bitcoin address

Contains the coinbase and block fee

Coinbase Reward and Fees



Calculation of Fees

are calculated to add to the coinbase transaction

Formula: Total Fees = Sum(Inputs) - Sum(Outputs)



Reward as new bitcoins

Based on block height

0-210 000: 50 Bitcoins (5 billion satoshis)

Reduced 50% every 210 000 blocks



Function to show calculation

GetBlockSubsidy in the Bitcoin Core client

Coinbase Reward and Fees



64 Halving

The maximum number of halvings allowed is 64

zero reward if the 64 halvings is exceeded



nSubsidy

the coinbase reward



Miner’s earning

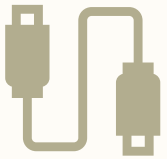
Total Fees + The Coinbase reward

Structure of the Coinbase Transaction



Coinbase Transaction Format

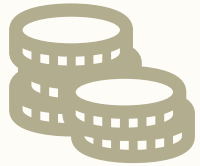
the coinbase transaction has a special format.



Input in Coinbase Transaction

Instead of a transaction input specifying a previous UTXO to spend

it has a “coinbase” input.

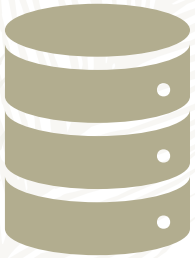


No UTXO reference In a coinbase transaction,

the first two fields are set first field is filled with 32 bytes all set to zero.

Instead of a “transaction hash” UTXO reference

Structure of the Coinbase Transaction



The “output index”

is filled with 4 bytes all set to 0xFF (255 decimal).



The “Unlocking Script” (scriptSig)

is replaced by coinbase data,

a data field used by the miners

Table 10-1. The structure of a "normal" transaction input

Size	Field	Description
32 bytes	Transaction Hash	Pointer to the transaction containing the UTXO to be spent
4 bytes	Output Index	The index number of the UTXO to be spent, first one is 0
1–9 bytes (VarInt)	Unlocking-Script Size	Unlocking-Script length in bytes, to follow
Variable	Unlocking-Script	A script that fulfills the conditions of the UTXO locking script
4 bytes	Sequence Number	Currently disabled Tx-replacement feature, set to 0xFFFFFFFF

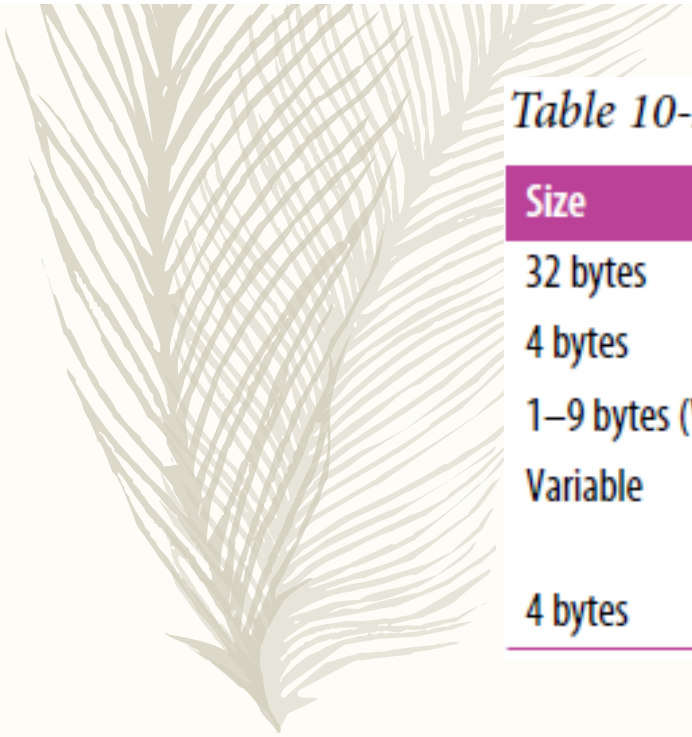


Table 10-2. The structure of a coinbase transaction input

Size	Field	Description
32 bytes	Transaction Hash	All bits are zero: Not a transaction hash reference
4 bytes	Output Index	All bits are ones: 0xFFFFFFFF
1–9 bytes (VarInt)	Coinbase Data Size	Length of the coinbase data, from 2 to 100 bytes
Variable	Coinbase Data	Arbitrary data used for extra nonce and mining tags. In v2 blocks; must begin with block height
4 bytes	Sequence Number	Set to 0xFFFFFFFF

Coinbase Data



No unlocking script

Coinbase transactions do not have

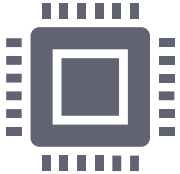
an unlocking script (aka, scriptSig) field.



Replaced field by coinbase

İt is replaced by coinbase data,

which must be between 2 and 100 bytes.



The rest is arbitrary data

Except for the first few bytes,

the rest of the coinbase data can be used by miners

in any way they want; it is arbitrary data.

Coinbase Data

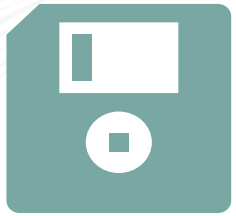
In the genesis block,

- Satoshi Nakamoto added the text
- “The Times 03/Jan/2009 Chancellor on brink of second bailout for banks” in the coinbase data,
- using it as a proof of the date and to convey a message.

Current usage of coinbase data

- to include extra nonce values and strings
- identifying the mining pool.

BIP-34



The first few bytes of the coinbase

used to be arbitrary,

but that is no longer the case.



As per BIP-34,

version-2 blocks (blocks with the version field set to 2)

must contain the block height index

as a script “push” operation in the beginning of the coinbase field.

Example Coinbase

Example coinbase

- **03**443b04**0385840206**2f503253482f

03

- instructs the script execution engine
- to push the next three bytes onto the script stack

0x443b04,

- are the block height encoded in little-endian format
- Reverse the order of the bytes and the result is
- 0x043b44, which is 277,316 in decimal.

0385840206

- used to encode an extra *nonce*

2f503253482f

- is the ASCII-encoded string /P2SH/,
- the mining node supports the P2SH improvement defined in BIP-16



Constructing the Block Header

Constructing the Block Header

- To construct the block header, the mining node needs to fill in six fields

Size	Field	Description
4 bytes	Version	A version number to track software/protocol upgrades
32 bytes	Previous Block Hash	A reference to the hash of the previous (parent) block in the chain
32 bytes	Merkle Root	A hash of the root of the merkle tree of this block's transactions
4 bytes	Timestamp	The approximate creation time of this block (seconds from Unix Epoch)
4 bytes	Target	The Proof-of-Work algorithm target for this block
4 bytes	Nonce	A counter used for the Proof-of-Work algorithm

Example Header

Version

- 0x02000000: version 2,
- encoded in little-endian format in 4 bytes

Previous Block Hash

- 0000000000000002a7bbd25a417c0374cc55261021e8a9ca74442b01284f0569

The root of Merkle Tree

- all the transactions are summarized
- C91c008c26e50763e9f548bb8b2fc323735f73577effbc55502c51eb4cc7cf2e

4-byte timestamp

- 1388185914: Friday, 27 Dec 2013, 23:11:54 UTC/GMT

the target,

- **0x19**03a30c: 0x19 is a hexadecimal exponent, 0x03a30c, is the coefficient
- defines the required Proof-of-Work to make this a valid block.
- 1-byte exponent, followed by a 3-byte mantissa (coefficient).

Nonce

- Initially set to 0

Ready to Mine

With all the other fields filled

- the block header is now complete
- and the process of mining can begin.

The goal is

- to find a value for the nonce
- results in a block header hash less than the target.

The mining node will need

- to test billions or trillions of nonce values
- before a nonce is found that satisfies the requirement.



Mining the Block

Proof-of-Work Algorithm

Target Representation

Retargeting to Adjust Difficulty

Mining the Block

Hash function SHA256

- is the function used in bitcoin’s mining process

Mining process

- is hashing the block header repeatedly,
- changing one parameter,
- until the resulting hash matches a specific target

The has function’s result

- cannot be determined in advance,

No pattern be created

- to produce a specific hash value.

Trying again and again

the only way to produce a hash result matching a specific target is

- to try again and again, randomly modifying the input
- until the desired hash result appears by chance.

Proof-of-Work Algorithm

A hash algorithm

- takes an arbitrary-length data input
- produces a fixed-length deterministic result,
- a digital fingerprint of the input

The resulting hash

- will always be the same
- can be easily calculated and verified
- by anyone implementing the same hash algorithm.

The key characteristic of a cryptographic hash algorithm is

- it is computationally infeasible
- to find two different inputs
- produce the same fingerprint known as a *collision*

Virtually impossible

- to select an input in such a way as
- to produce a desired fingerprint,
- other than trying random inputs

Usage of Nonce

SHA256, the output is

- always 256 bits long, regardless of the size of the input

Nonce

- The number used as a variable
- to change the SHA 256 result is called a *nonce*
- if the output of the hash function is evenly distributed we would expect to find a result with a 0 as the hexadecimal prefix once every 16 hashes (one out of 16 hexadecimal digits 0 through F
- In numerical terms, that means finding a hash value that is less than
- 0x1000000000000000000000000000000000000000000000000000000000000000

this threshold is called the *target*

- the goal is to find a hash that is numerically less than the target

Analogy

imagine a game where players throw a pair of dice repeatedly, trying to throw less than a specified target.

In the first round, the target is 12

Unless you throw double-six, you win. In the next round the target is 11.

Players must throw 10 or less to win, again an easy task.

Let’s say a few rounds later the target is down to 5.

Now, more than half the dice throws will exceed the target and therefore be invalid.

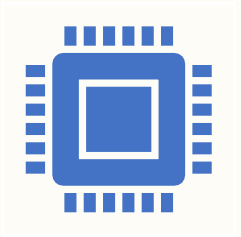
It takes exponentially more dice throws to win, the lower the target gets.

Eventually, when the target is 2 (the minimum possible), only one throw out of every 36, or 2% of them, will produce a winning result.

Proof of Work



one can estimate the amount of work it takes to succeed from the difficulty imposed by the target.



When the algorithm is a based on a deterministic function such as SHA256, the input itself constitutes *proof* that a certain amount of *work* was done to produce a result below the target.



Hence, *Proof-of-Work*.

Proof of Work

the winning “nonce” is 13 and this result can be confirmed by anyone independently.

Anyone can add the number 13 as a suffix to the phrase “I am Satoshi Nakamoto” and compute the hash, verifying that it is less than the target.

The successful result is also Proof-of-Work, because it proves we did the work to find that nonce.

While it only takes one hash computation to verify, it took us 13 hash computations to find a nonce that worked.

If we had a lower target (higher difficulty) it would take many more hash computations to find a suitable nonce, but only one hash computation for anyone to verify.

Proof-of-Work vs. Target

- The Proof-of-Work must produce a hash that is *less than* the target.
- A higher target means it is less difficult to find a hash that is below the target.
- A lower target means it is more difficult to find a hash below the target.
- The target and difficulty are inversely related.

Mining Steps

The miner constructs a candidate block filled with transactions.

Next, the miner calculates the hash of this block's header and sees if it is smaller than the current *target*.

If the hash is not less than the target, the miner will modify the nonce (usually just incrementing it by one)

and try again.

Difficulty and Search Space

At the current difficulty in the bitcoin network

- miners have to try quadrillions of times
- before finding a nonce that results in
- a low enough block header hash

increasing the difficulty by 1 bit

- causes a doubling in the time it takes to find a solution.

the entire 256-bit number space,

- each time you constrain one more bit to zero,
- you decrease the search space by half.

Network's Processing Capacity

to find a block whose header hash is less than:

- 00000000000000029AB9000000000000000000000000000000000000

1.8 septa-hashes

- It will take on average more than 1.8 septa-hashes
- thousand billion billion hashes
- per second for the network to discover the next block.

3 exa-hashes

- the network is bringing 3 exa-hashes per second (EH/sec) of processing power to bear
- to find a block in about 10 minutes on average.

The Target

Notation expresses

- the Proof-of-Work target as a coefficient/exponent format,
- with the first two hexadecimal digits for the exponent
- and the next six hex digits as the coefficient.

Formula

- target = coefficient * 2^(8 * (exponent – 3))

Example

- **0x19**03a30c
- the exponent is 0x19 and the coefficient is 0x03a30c
- target = 0x03a30c * 2^(0x08 * (0x19 - 0x03))
- = 0x03a30c * 2^0xB0^
- target = 0x0000000000000003A30C000000000000000000000000000000000000000000
- Decimal: 238,348 * 2^176

Miner's Performance



60 leading bits

set to zero in binary



With this level of difficulty,

a single miner processing 1 TH/sec

would only find a solution once every 8,496 blocks

or once every 59 days, on average

Retargeting to Adjust Difficulty



Critical questions

Why is the difficulty adjustable,

who adjusts it,

how?



Bitcoins hearthbeat

Bitcoin’s blocks are generated

every 10 minutes, on average



Constant frequency

The frequency has to remain constant

not just over the short term, but over a period of many decades

Variable of Frequency

Variables affecting the frequency,

- computer power will continue to increase at a rapid pace.
- the number of participants in mining
- the computers used in mining will also constantly change

To keep the block generation time at 10 minutes,

- the difficulty of mining must be adjusted
- to account for these changes

Adjustment of target

Target is a dynamic parameter

- periodically adjusted to meet a 10-minute block interval goal

Retargeting occurs

- automatically and on every node independently

Every 2,016 blocks,

- all nodes retarget the Proof-of-Work

Equation

- New Target = Old Target * (Actual Time of Last 2016 Blocks / 20160 minutes)

Adjustment of target



The equation for retargeting

measures the time it took to find the last 2,016 blocks

and compares that to the expected time of 20,160 minutes



increase the difficulty

If the network is finding blocks faster than every 10 minutes,

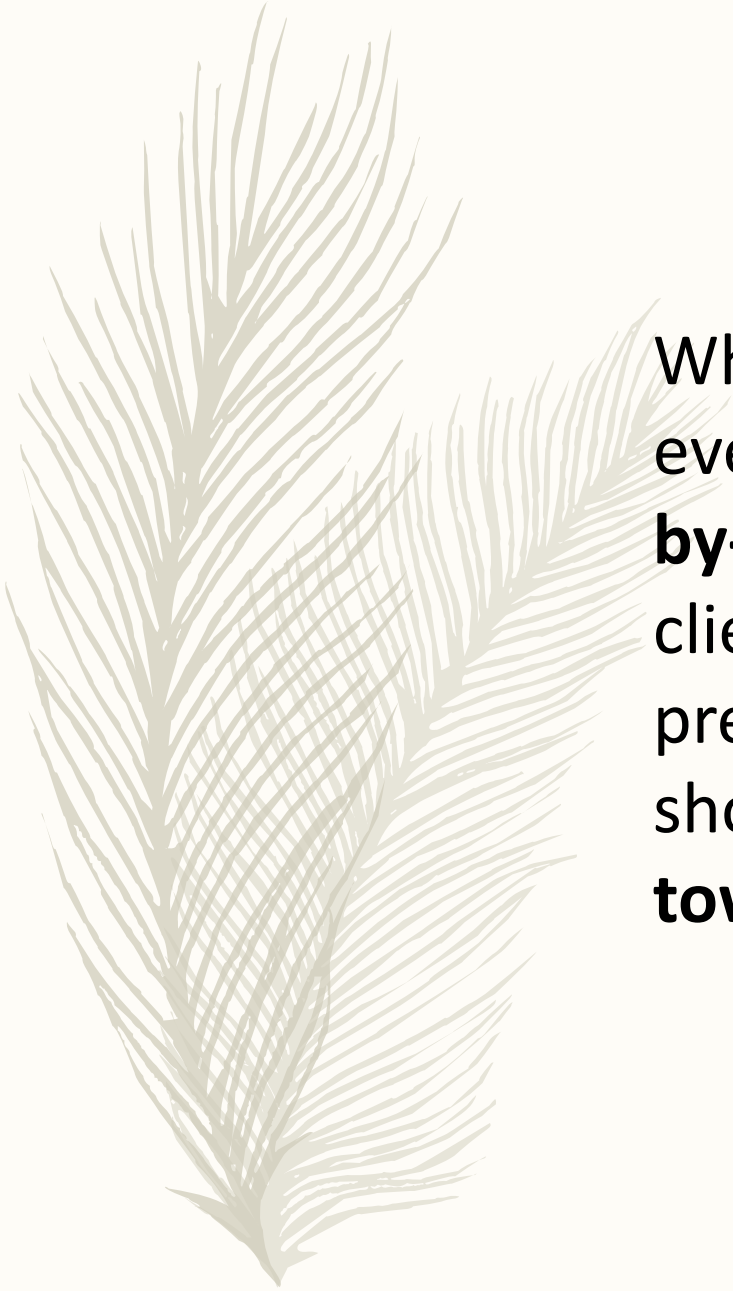
the difficulty increases (target decreases).



Decrease the difficulty

If block discovery is slower than expected,

the difficulty decreases (target increases)



While the target calibration happens every **2,016** blocks, because of an **off-by-one error** in the original Bitcoin Core client it is based on the total time of the previous **2,015** blocks (not 2,016 as it should be), resulting in a retargeting **bias toward higher difficulty** by **0.05%**.



Successfully Mining the Block

Successfully Mining a Block

the hardware mining machines

- run the SHA256 algorithm in parallel at incredible speeds
- find a solution and sends it back to the mining node

When inserted into the block header,

- the nonce produces a block hash
- which is less than the target

Successfully Mining a Block

mining node transmits the block to all its peers

- They receive, validate, and then propagate the new block.

As the block ripples out across the network,

- each node adds it to its own copy of the blockchain,
- extending it to a new height

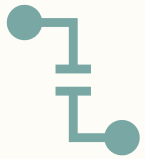
Other miners

- Stop mining the block and start for the next block



Validating a New Block

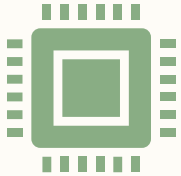
Validating a New Block



Independent validation

New block must independently be validated

by every node on the network.



Validation method

each node performs a series of tests

to validate it

before propagating it to its peers.



Valid blocks propagate

This ensures

only valid blocks are propagated

on the network.

Validating a New Block

Honest miners

- The independent validation also ensures
- miners act honestly

Rejection

- Unhonest miners blocks are rejected

Harm to unhonest miners

- lose the reward
- waste the effort to find a Proof-of-Work solution

Validating a New Block

Perfect block

- The miners have to construct
- a perfect block
- mined with correct Proof-of-Work algorithm

Base

- based on the shared rules

Common practice

- all nodes follow

Block Validation Criteria

Sytactic validation

- The block data structure
- is syntactically valid

Valid hashing

- The block header hash
- is less than the target
- enforces the Proof-of-Work

Two hourse rule

- The block timestamp is less than
- two hours in the future
- allowing for time errors

Block Validation Criteria

Block size

- The block size is
- within acceptable limits

Coinbase transaction

- The first transaction
- and only the first
- is a coinbase transaction

Transactions are valid

- All transactions within the block are valid
- using the transaction checklist



Assembling and Selecting Chains of Blocks

Blockchain Forks

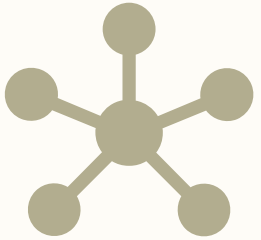
Assembling and Selecting Chains of Blocks



The final step in decentralized consensus

assembly of blocks into chains

the selection of the chain with the most Proof-of-Work



Block sets

Nodes maintain three sets of blocks:

Assembling and Selecting Chains of Blocks

Primary chains

- those connected to the main blockchain

Secondary chains

- those that form branches off the main blockchain

orphans

- blocks that do not have a known parent in the known chains

Main Chain

valid chain of blocks

- has the most cumulative Proof-of-Work
- associated with it.

The main chain

- will also have branches
- with blocks
- that are “siblings” to the blocks on the main chain

They are kept

- for future reference
- in case one of those chains
- is extended to exceed the main chain in work

Main Chain

Slot new block

- When a new block is received,
- a node will try
- to slot it into the existing blockchain

Look for previous block hash

- The node will look at the
- block’s “previous block hash” field

Main Chain

Find parent

- the node will attempt
- to find that parent in the existing blockchain

Extension

- Most of the time, the parent will be the “tip” of the main chain,
- meaning this new block extends the main chain

Orphan Blocks

Orphan block

- A block with no parent

How to happen?

- Occurs when two blocks were mined within short of time
- Arrived in reverse order that is child before parent

Orphan Blocks

orphan block pool

- Saved until the parent is received

When parent is received

- the orphan can be pulled out of the orphan pool
- and linked to the parent

Blockchain Forks

Inconsistency

- the blockchain is a decentralized data structure
- different copies of it are not always consistent

Different perspectives of the blockchain

- Blocks arrive at different nodes and times

Blockchain Forks

Selection

- each node always selects and attempts
- to extend the longes chain

the longest chain

- represents the most Proof-of-Work

Calculation

- calculate the total amount of work
- by summing the work recorded in each block in a chain

Forks

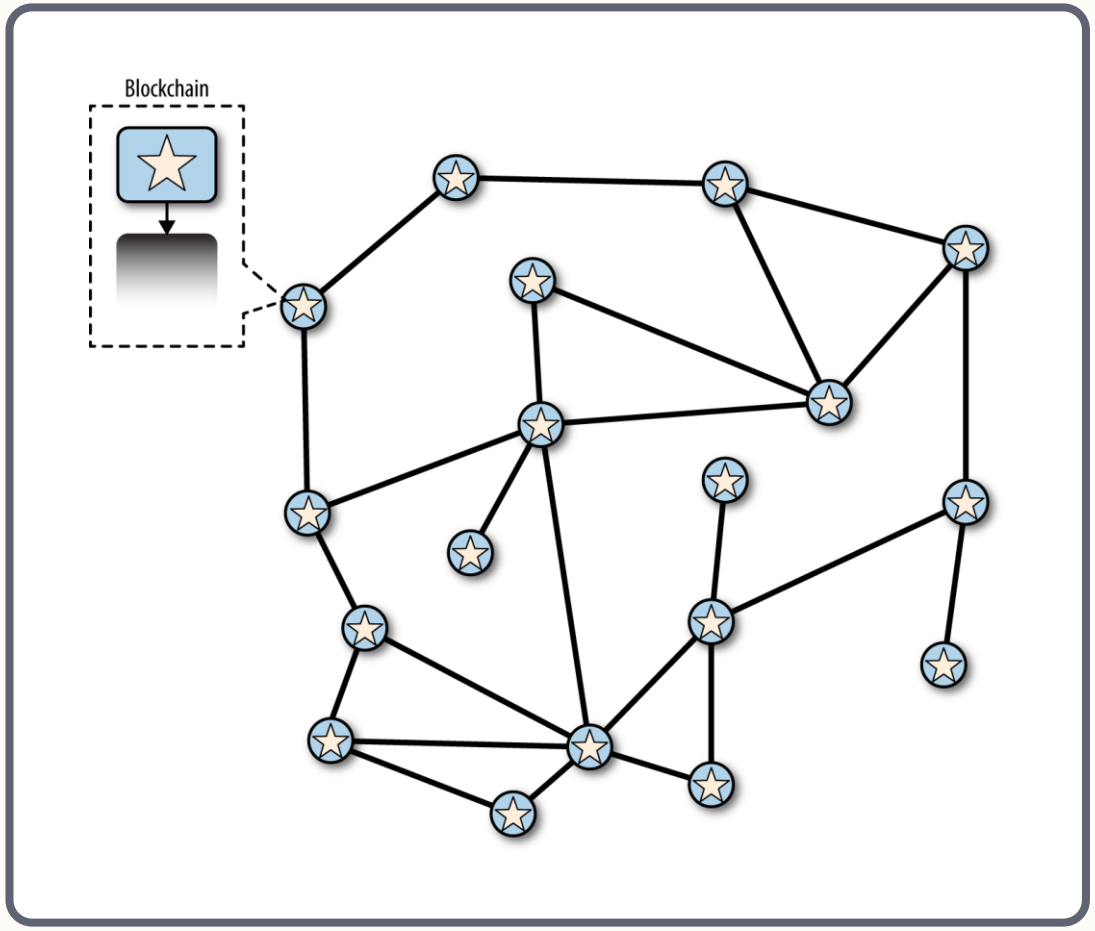


Figure 10-2. Before the fork—all nodes have the same perspective

Temporary inconsistency

- occur as temporary inconsistencies
- between versions of the blockchain

Transmission delays

- occur naturally as a result of
- transmission delays in the global network

Forks

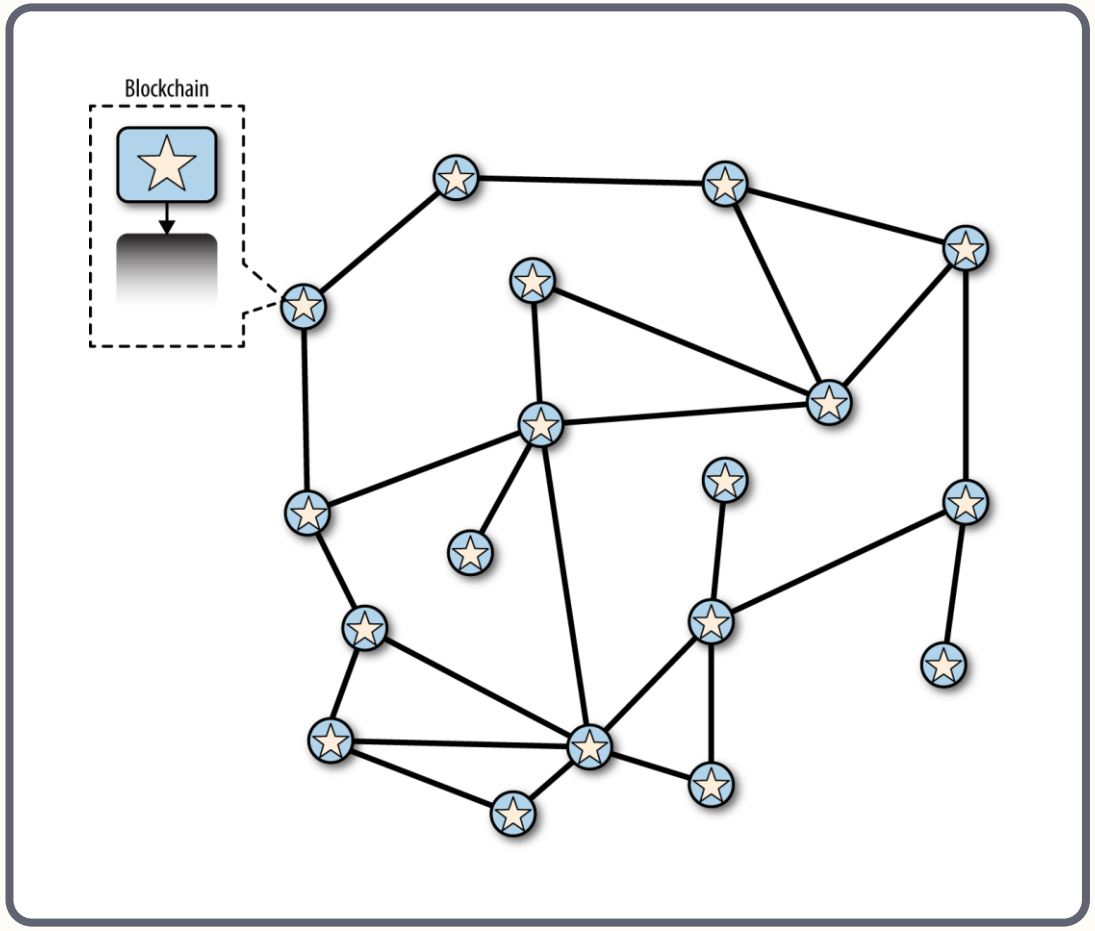


Figure 10-2. Before the fork—all nodes have the same perspective

Simulation solutions

- Happens when two distant miners
- simultaneously solve different candidate blocks

Multiple chains

- If arriving block's parent has another child
- They are connected to secondary chain

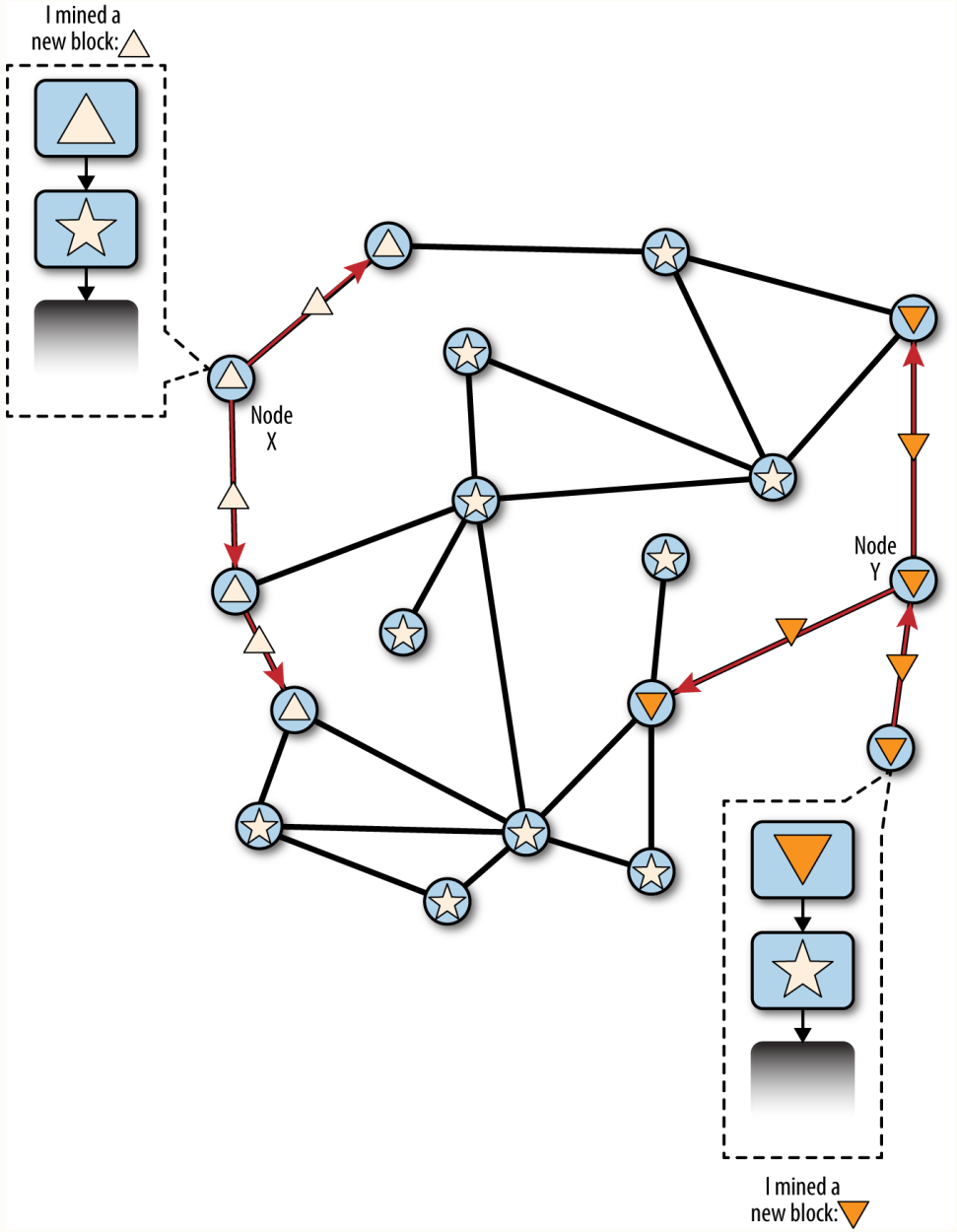


Figure 10-3. Two blocks found simultaneously

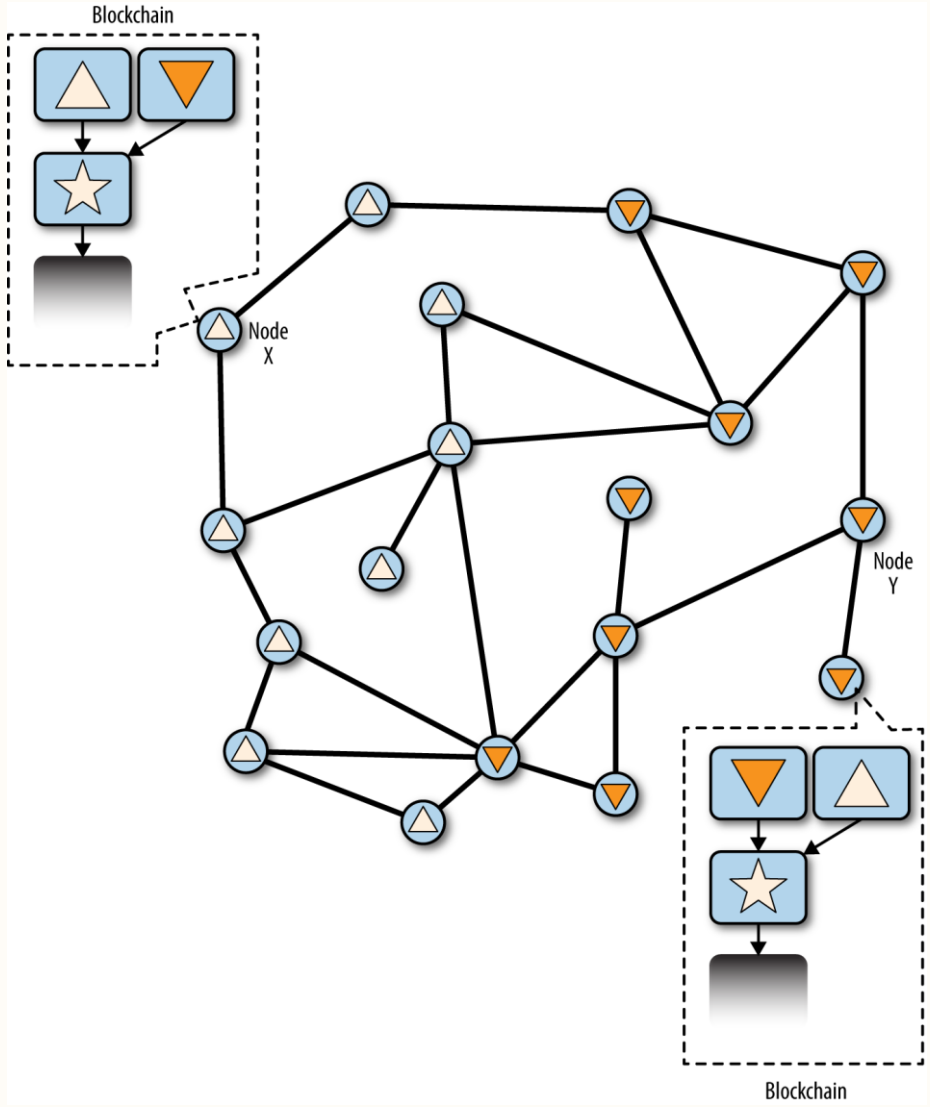
Propagation of Two Blocks Simultanously

Two simultanous blocks

- Two nodes mine two blocks simultanously
- Two possible blocks are propagated

Two Perspectives

- Some part of network is unaware of the other block



Receive of Other Block

Losing chain

- Blocks later arrived are linked to secondary chain

Two correct chains

- Neither is correct nor incorrect

Win of losing chain

- One of the miners will find a solution to next block
- The first chain finding next block wins

The Winner Chain

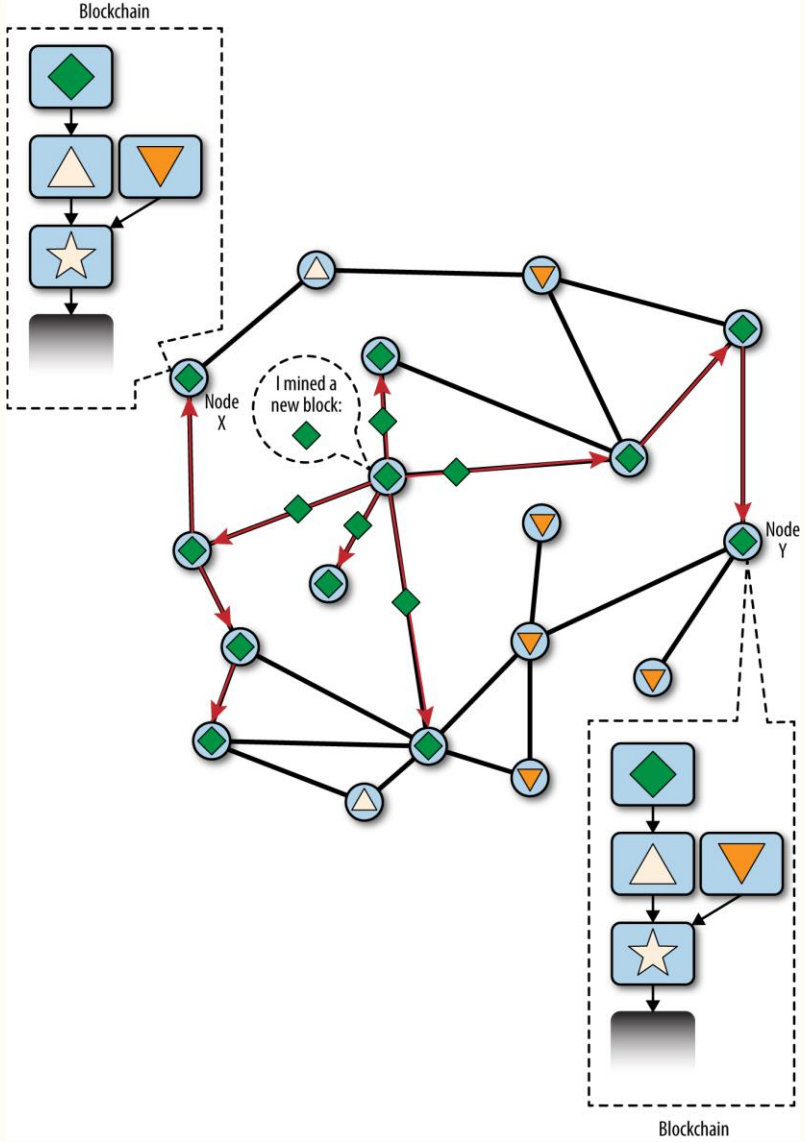


Figure 10-5. a new block extends one fork, reconverging the network

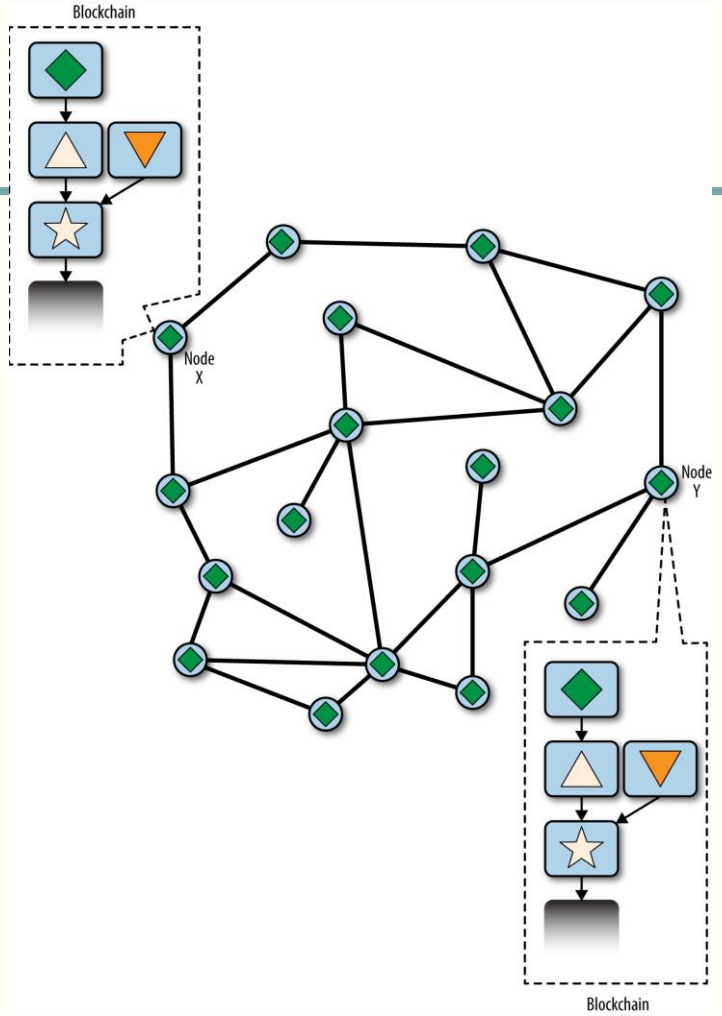


Figure 10-6. Visualization of a blockchain fork event: the network reconverges on a new longest chain

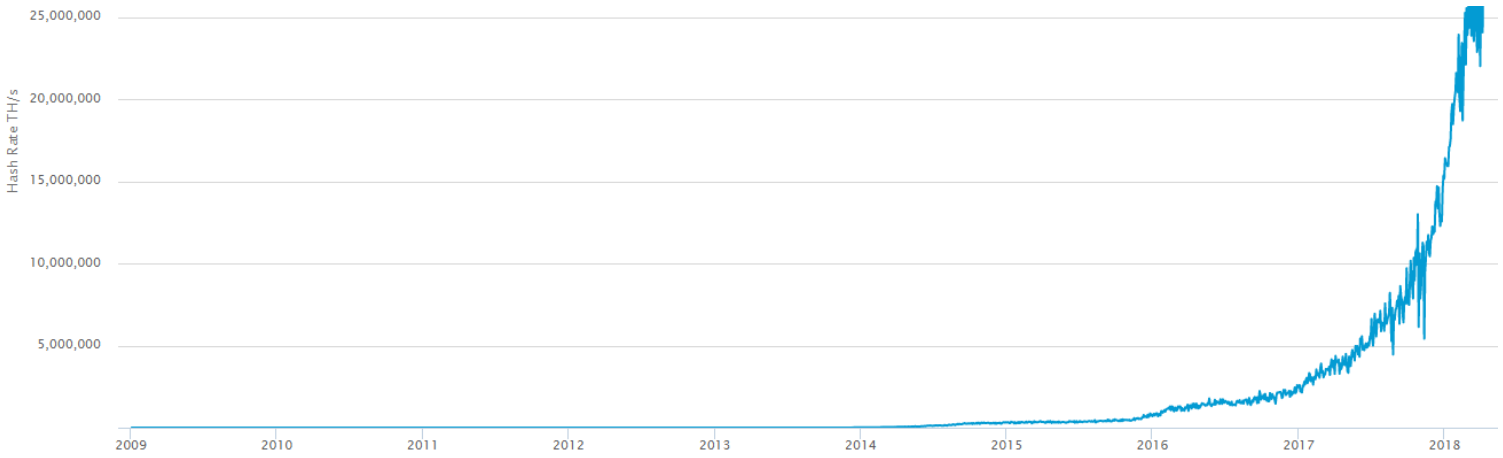


Mining and the Hashing Race

The Extra Nonce Solution

Mining Pools

Mining and the Hashing Race

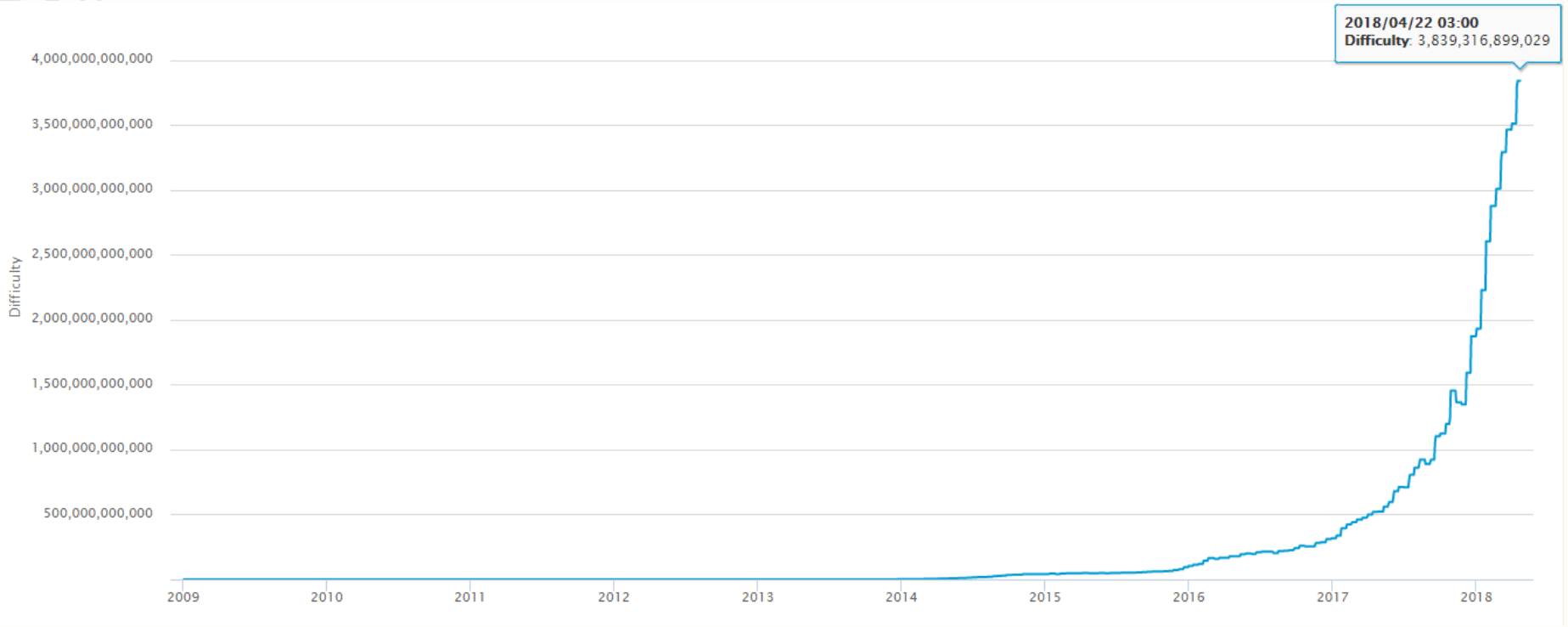


total hashing power of the bitcoin network

- 2009: 0.5 MH/sec–8 MH/sec (16× growth)
- 2010: 8 MH/sec–116 GH/sec (14,500× growth)
- 2011: 16 GH/sec–9 TH/sec (562× growth)
- 2012: 9 TH/sec–23 TH/sec (2.5× growth)
- 2013: 23 TH/sec–10 PH/sec (450× growth)
- 2014: 10 PH/sec–300 PH/sec (3000× growth)
- 2015: 300 PH/sec-800 PH/sec (266× growth)
- 2016: 800 PH/sec-2.5 EH/sec (312× growth)
- 2017: 2.5 EH/sec - 15 EH/Sec (6x growth)
- 23.04.2018: 15 EH/Sec - 29 EH/sec (0.9x growth)

Difficulty

- Hashing power
  - As the amount of hashing power applied
  - to mining bitcoin has exploded,
- Difficulty
  - the difficulty has risen to match it.



The Extra Nonce Solution

As difficulty increased

- miners often cycled
- through all 4 billion values
- of the nonce without finding a block.

Resolution

- this was easily resolved
- by updating the block timestamp

The Extra Nonce Solution

hardware's power

- Once mining hardware exceeded 4 GH/sec
- Time-stamp based approach
- became increasingly difficult

Speed of exhausting

- because the nonce values
- were exhausted in less than a second

The Extra Nonce Solution

The solution

- to use the coinbase transaction
- as a source of extra nonce values

Space for coinbase

- coinbase script can store
- between 2 and 100 bytes of data

Extra nonces

- 8 bytes of extra nonce
- + 4 bytes of “standard” nonce

The Extra Nonce Solution

8 extra bytes

- allow miners
- to explore a total 2^{96} possibilities *per second*

What is 2^{96}?

- 8 followed by 28 zeros
- without having to modify the timestamp.

Mining Pools

Chance of solo miner

- individual or solo miners working alone
- don’t stand a chance

Chance with ASIC systems

- Even the fastest consumer ASIC mining system
- cannot keep up with commercial systems

Competition with commercials

- stack tens of thousands of these chips
- in giant warehouses
- near hydroelectric powerstations.



Tech Hustler — How To Mine Bitcoin At Home In 2021
https://www.youtube.com/watch?v=bJWyBY78q48



Glame TOP 5 — Top 5 Largest BITCOIN MINES on Earth
https://www.youtube.com/watch?v=pWnN3WQ2m5M

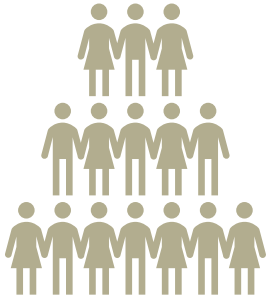
Mining Pools



Collaboration of pools

Miners now collaborate

to form mining pools



Unite to compete

Miners combine their hashing power

sharing the reward among thousands of participants.

Result of Mining Pools



Small share

miners get a smaller share

of the overall reward



Reduced uncertainty

get rewarded in shorter periods

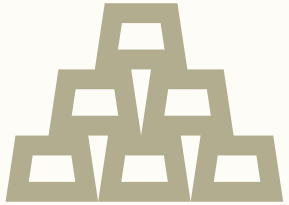
reducing uncertainty

Coordination of Pools



Size of pools

Mining pools coordinate hundreds or thousands of miners



Protocols

Pools use over specialized pool-mining protocols.

Coordination of Pools



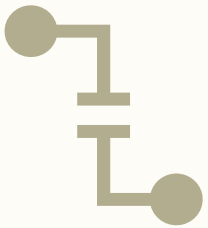
Connecting to the pool

The individual miners

configure their mining equipment

to connect to a pool server

after creating an account with the pool.



Continues connection

mining hardware remains connected to the pool server

while mining synchronizing their efforts

with the other miners

Coordination of Pools



Sharing

When someone in the pool successfully mines a block,

the reward is earned by the pool

then shared with all miners

in proportion to the number of shares

they contributed to the effort.



Paying miners

The pool server will periodically make payments

to the miners' bitcoin addresses,

once their share of the rewards

has reached a certain threshold.



Charge

Typically, the pool server charges

a percentage fee of the rewards

Variation of Miners in Pool



Variation in size

Pools are open

to any miner

big or small

professional or amateur.



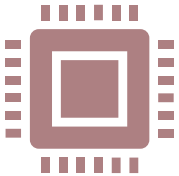
Variation in devices

have some participants

with a single small mining machine

others with a garage full of

high-end mining hardware



Variation in consumption

Some will be mining

with a few tens of a kilowatt of electricity,

others will be running a data center

consuming a megawatt of power.

Fair Distribution of Rewards



Fairly distribution of the rewards

use bitcoin’s Proof-of-Work algorithm

to measure each pool miner’s contribution,



set at a lower difficulty

so that even the smallest pool miners win a share frequently enough

to make it worthwhile to contribute to the pool

Fair Distribution of Rewards



Fairly distribution of the rewards

use bitcoin’s Proof-of-Work algorithm

to measure each pool miner’s contribution,



set at a lower difficulty

so that even the smallest pool miners win a share frequently enough

to make it worthwhile to contribute to the pool

Managed pools

Managed pool	there is a company or individual running a pool server
pool operator	owner of the pool server charges pool miners a percentage fee of the earnings

Managed pools

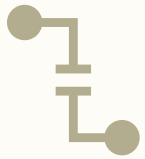


Coordination of miners

The pool server runs specialized software

and a pool-mining protocol

to coordinate the activities of the pool miners.



Access to the chain

The pool server is connected to one or more full bitcoin nodes

has direct access to a full copy of the blockchain database



Benefit

allows the pool server to validate blocks and transactions

on behalf of the pool miners

relieving them of the burden of running a full node

Mining Protocols

GetBlockTemplate (GBT)

- Older standard
- mostly obsolete since late 2012
- Doesn’t easily support hash rates over 4GH/s

Stratum (STM)

Mining Protocols

Mining protocol

- create block *templates*
- contain a template of a candidate block header

Candidate block construction

- aggregating transactions,
- adding a coinbase transaction (with extra nonce space),
- calculating the merkle root,
- linking to the previous block hash

Mining Protocols

Send the header

- The header of the candidate block is
- sent to each of the pool miners as a template

Miners

- Each pool miner then mines using the block template,
- at a higher (easier) target than the bitcoin network target
- sends any successful results back to the pool server to earn shares

Peer-to-peer mining pool (P2Pool)

Cheating possibility

- Managed pools create the possibility of
- cheating by the pool operator,

Consensus attack

- direct the pool effort
- to **double-spend** transactions or **invalidate** blocks

Single point-of-failure.

- If the pool server is down or is slowed by a denial-of-service attack,
- the pool miners cannot mine

Peer-to-peer mining pool (P2Pool)

Pool without central operator

- In 2011, to resolve these issues of centralization,
- a new pool mining method was proposed and implemented:
- P2Pool, a peer-to-peer mining pool without a central operator

Decentralization

- P2Pool works by decentralizing the functions of the pool server,
- implementing a parallel blockchain-like system called a *share chain*

Share chain

- A share chain is a blockchain running
- at a lower difficulty than the bitcoin blockchain

Peer-to-peer mining pool (P2Pool)

Decentralized pool

- The share chain allows pool miners
- to collaborate in a decentralized pool

30 blocks/sec

- share chain at a rate of one share block every 30 seconds.

Peer-to-peer mining pool (P2Pool)

Record share reward

- Each of the blocks on the share chain
- records a proportionate share reward

for contribution

- the pool miners who contribute work,

carry the shares

- forward from the previous share block.

Peer-to-peer mining pool (P2Pool)

Achieving network target

- When one of the share blocks
- also achieves the bitcoin network target
- it is propagated and included on the bitcoin blockchain
- rewarding all the pool miners
- who contributed to all the shares

instead of a pool server keeping track of pool miner shares and rewards,

- the share chain allows all pool miners
- to keep track of all shares
- using a decentralized consensus mechanism
- like bitcoin’s blockchain consensus mechanism.

Differences of P2Pool and Stratum



P2Pool mining

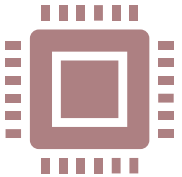
is more complex than pool mining



Reason

Computational

Hardware



Requirements

the pool miners run a dedicated computer

with enough disk space memory

Internet bandwidth

to support a full bitcoin node

the P2Pool node software.

Differences of P2Pool and Stratum



P2Pool miners

connect their mining hardware

to their local P2Pool node,



Simulate

to simulate the functions of a pool server

by sending block templates

to the mining hardware.



On P2Pool

individual pool miners

construct their own candidate blocks



Like solo miners

aggregating transactions

much like solo miners



Share mine

mine collaboratively

on the share chain



P2Pool is a hybrid approach

has the advantage of much more granular payouts

than solo mining

Pros & Cons

Less control

- to a pool operator
- like managed pools

P2Pool reduces

- the concentration of power
- by mining pool operators

Pros & Cons

Sensible against 51% attacks

- Vulnerable to 51% attacks
- against the share chain itself

Robustness

- P2Pool makes bitcoin more robust overall,
- as part of a diversified mining ecosystem



Consensus Attacks

Consensus Attacks

consensus mechanism

- depends on having a majority of the miners
- acting honestly out of self-interest

Attack of miners

- Miners can attack the consensus mechanism
- so as to disrupt the security and availability of the bitcoin network

Condition

- if miners can achieve a significant share of the mining power

Impact of Consensus Attack

Only future blocks are affected

- consensus attacks can only affect future consensus,
- or at best, the most recent past (tens of blocks).

old blocks practically immutable

- in theory, a fork can be achieved at any depth,
- in practice, the computing power needed to force a very deep fork is immense

Impact of Consensus Attack

Security is not affected

- Consensus attacks also do not affect
- the security of the private keys and signing algorithm (ECDSA).

No bitcoins can be stolen

- A consensus attack cannot steal bitcoin, spend bitcoin without signatures,
- redirect bitcoin, or otherwise change past transactions or ownership records

“51% attack”

Scenario

- miners, controlling a majority (51%)
- of the total network’s hashing power
- can attack bitcoin

Mine majority of the blocks

- can cause deliberate “forks”
- double-spend transactions

deliberate “forks”

the attacker causes previously confirmed blocks

- to be invalidated by forking below them
- and re-converging on an alternate chain.

Six blocks in practice

- an attacker can invalidate six or more blocks in a row
- causing transactions to be invalidated.

double-spend transactions



How to happen?

broadcasts a competing transaction

spends the same inputs (UTXO)

and cancels the payment



Two types

before a transaction is confirmed

or if the attacker takes advantage of a blockchain fork to undo several blocks.



Solution

Wait for 6 blocks to be confirmed

denial-of-service attacks

Denial transactions

- deny service
- to specific bitcoin participants
- ignore specific transactions

Forking to exclude

- If confirmed by another miner
- the attacker can deliberately fork that block,
- excluding the specific transactions

denial-of-service attacks

result in

- a sustained denial-of-service
- against a specific address or set of addresses

30%

- Various types of consensus attacks are possible
- with as little as 30% of the hashing power

Protection

Evolution

- the bitcoin network and software
- are constantly evolving,

Countermeasures of bitcoin community

- consensus attacks would be met
- with immediate countermeasures
- by the bitcoin community

Strategy

- making bitcoin hardier
- Stealthier
- more robust than ever.



Changing the Consensus Rules

Hard Forks

Hard Forks: Software, Network, Mining, and Chain

Diverging Miners and Difficulty

Contentious Hard Forks

Soft Forks

Criticisms of Soft Forks

Changing the Consensus Rules



The rules of consensus

determine the validity of transactions and blocks

are the basis for collaboration between all bitcoin nodes

are responsible for convergence of local perspectives into a single consistent blockchain



consensus rules vary in time In long term

to accommodate

new features, improvements, or bug fixes



upgrades to a consensus system

are much more difficult

require coordination between all the participants

Hard Forks

a change in the consensus rules

- network may diverge into two chains
- after the fork the network does not reconverge onto a single chain
- Two chains evolve independently

occur

- when part of the network is operating under a different set of consensus rules
- than the rest of the network

because of

- a bug or
- a deliberate change in the implementation of the consensus rules

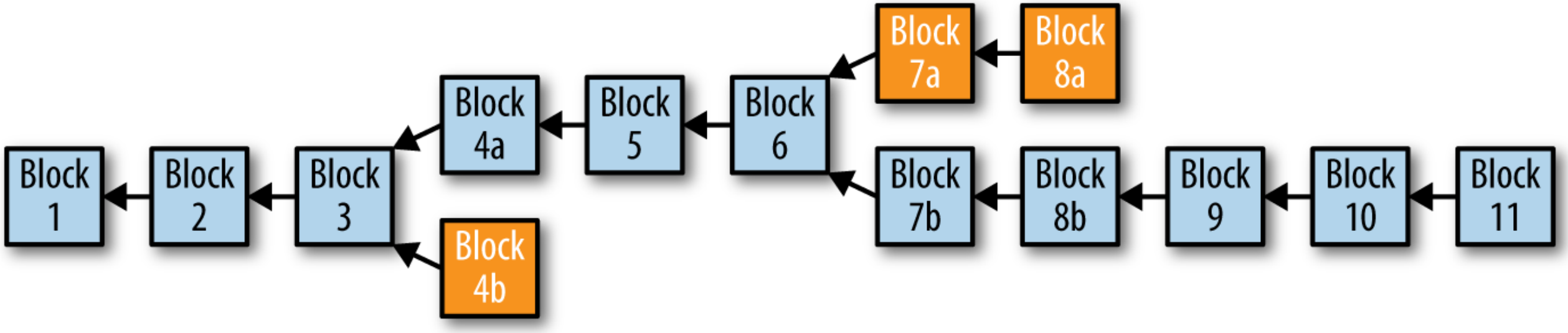


Figure 10-9. A blockchain with forks

Forward Compatibility



Any nodes that do not upgrade to the new consensus rules

are unable to participate in the consensus mechanism

and are forced onto a separate chain at the moment of the hard fork.



Thus, a change introduced by a hard fork

can be thought of as **not** “forward compatible,”



nonupgraded systems

can no longer process the new consensus rules.

Hard Forks vs. Soft Forks

Circumstances lead to a hard fork:

- a bug in the consensus rules
- and a deliberate modification of the consensus rules

Software forks

- a new software implementation of the consensus rules
- to be developed, adopted, and launched

Not all software forks result in hard fork

- Alternative implementations of Bitcoin Core,
- software forks, not changing the consensus rules and barring a bug

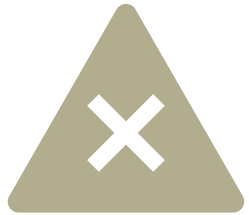
Changing Consensus Rules



in the validation of transactions or blocks



apply to bitcoin scripts or cryptographic primitives such as digital signatures



because of implicit consensus constraints imposed by system limitations or implementation details

Stages of Hardfork



Software fork

an alternative implementation of the client, with modified consensus rules, is created by developers



Network fork

a certain percentage of miners, wallet users, and intermediate nodes

may adopt and run this implementation



Mining fork

Once a miner using the new rules mines a block, the mining power and chain will also fork.

New miners will mine on top of the new block,

while old miners will mine a separate chain based on the old rules.



Chain fork

A single transaction or block based on the new rules will ripple through the network

and result in the partition into two networks

Diverging Miners and Difficulty

As miners diverge

- into mining two different chains,
- the hashing power is split between the chains.

Example scenario

- 80%–20% split

Diverging Miners and Difficulty

80% chain

- 20% of loss in mining power
- Mining: 12 Minuts/Block
- 2016 blocks, 24,192 minutes, 16.8 days
- the difficulty will adjust (reduced by 20%)

20% chain

- 80% of loss in mining power
- Mining: 50 Minuts/Block
- 2016 blocks, 100,800 minutes, 10 weeks
- the difficulty will adjust (reduced by 80%)

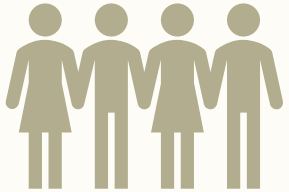
Contentious Hard Forks



bitcoin development roadmap,

we will see new development tools,

practices, methodologies, and communities emerge



Vision for hard forks

Various community members have different vision

Contentious Hard Forks

Risky

- risky and to be avoided since force a minority
- to either upgrade or remain on a minority chain.

Essential

- for upgrading the consensus rules
- provides a clean break with the past.

Rarely used

- with a lot of advance planning
- and only under nearunanimous consensus.

Soft Forks



In practice

a soft fork is not a fork at all.



A soft fork is a forward-compatible change

to the consensus rules that allows unupgraded clients

to continue to operate in consensus with the new rules



soft fork upgrades can only be used

to constrain the consensus rules,

not to expand them.

Implementation Methods



Soft forks redefining NOP opcodes

based on the reinterpretation of NOP opcodes

Bitcoin Script had ten opcodes reserved for future use, NOP1 through NOP10

A soft fork therefore can modify the semantics of a NOP code to give it new meaning



Other ways to soft fork upgrade

fork mechanism was introduced that does not rely on NOP

In future other methods might be introduced



Example

move the unlocking script (witness) from inside the transaction

to an external data structure

Criticisms of Soft Forks



Technical debt

soft forks are more technically complex

cost of code maintenance

likelihood of bugs and security vulnerabilities



Validation relaxation

Unmodified clients see transactions as valid,

without evaluating the modified consensus rules



Irreversible upgrades

If a soft fork upgrade were to be reversed after being activated,

transactions created under the new rules

could result in a loss of funds under the old rules.



Soft Fork Signaling with Block Version

BIP-34 Signaling and Activation

BIP-9 Signaling and Activation

Soft Fork Signaling with Block Version

Majority of miners must agree

- they are ready and willing
- to enforce the new consensus rules

To coordinate the operation

- there is a signaling mechanism
- to show the support for a consensus rule change

mechanism was introduced

- with the activation of BIP-34 in March 2013
- and replaced by the activation of BIP-9 in July 2016.

BIP-34 Signaling and Activation



used the block version field

to allow miners to signal readiness

for a specific consensus rule change



To signal the change and activation of BIP-34,

miners set the block version to “2” instead of “1”

BIP-34 Signaling and Activation: Rules



If 75%

750 of the most recent 1000 blocks are marked with version “2,”

Then version “2” blocks must contain block height in the coinbase transaction

or they are rejected as invalid.

Version “1” blocks are still accepted



When 95% (950 of the most recent 1000 blocks) are version “2,”

version “1” blocks are no longer considered valid.

Version “2” blocks are valid only if they contain the block-height in the coinbase

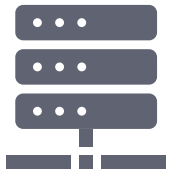
Other Usages of Activation



BIP-66 Strict DER Encoding of Signatures

was activated by BIP-34 style signaling

with a block version “3” and invalidating version “2” blocks



BIP-65 CHECKLOCKTIMEVERIFY

was activated by BIP-34 style signaling

with a block version “4” and invalidating version “3” blocks



Retire of BIP-34

After the activation of BIP-65,

the signaling and activation mechanism of BIP-34 was retired

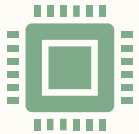
and replaced with the BIP-9 signaling mechanism

BIP-9 Signaling and Activation



Limitations of mechanism

used by BIP-34, BIP-66, and BIP-65



By using the integer value of the block version,

only one soft fork could be activated at a time



the mechanism didn't offer a straightforward way

to reject a change and then propose a different one

since the block version was incremented



Each new change irrevocably reduced

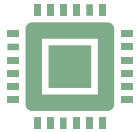
the available block versions for future changes.

BIP-9 Features



BIP-9 interprets the block version

as a bit field instead of an integer



1-4 are already been used

only 29 bits remain available to be used as a bit field



BIP-9 also sets

a maximum time for signaling and activation



the proposal is considered rejected

If a proposal is not activated within the TIMEOUT period

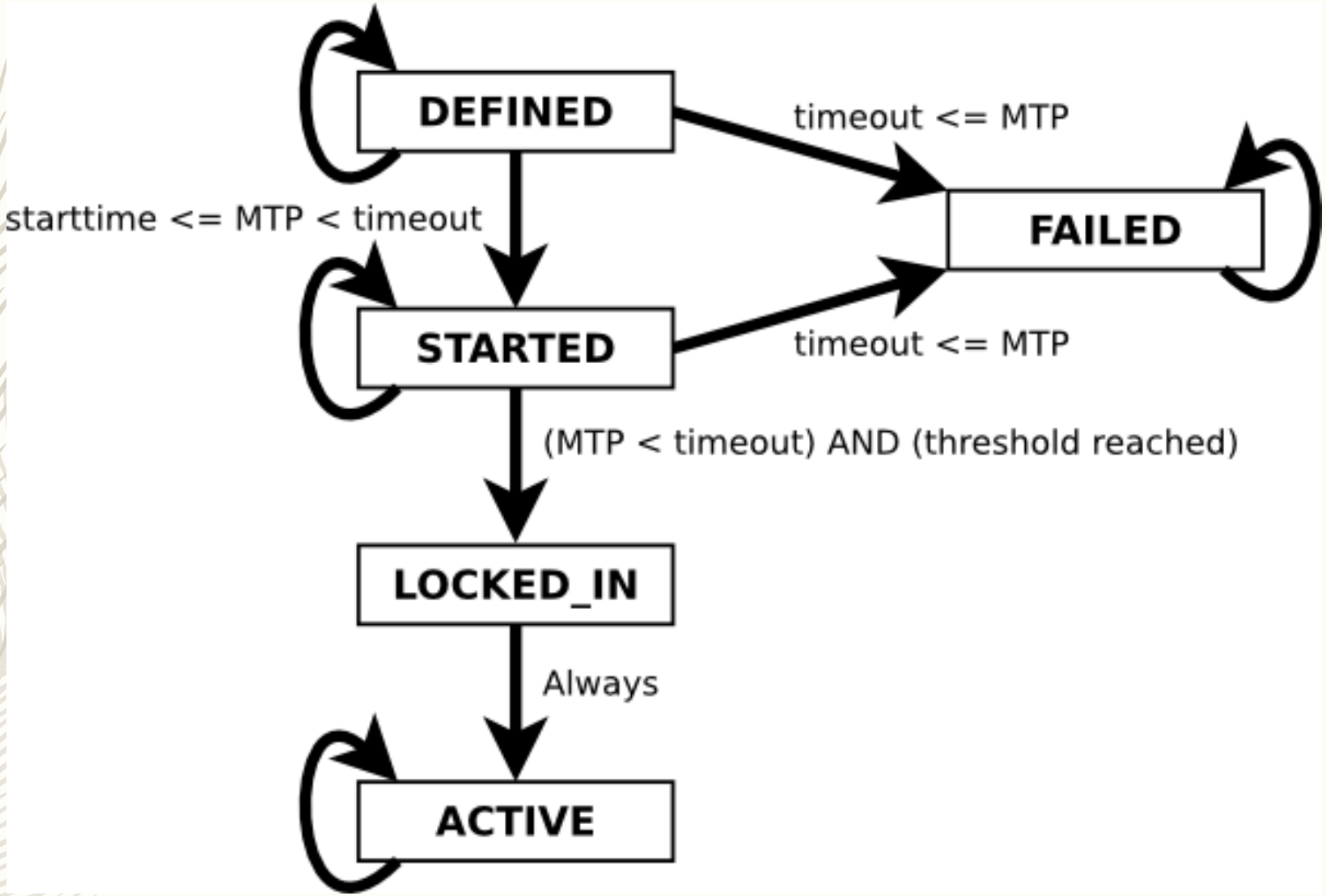


Figure 10-10. BIP-9 state transition diagram



Consensus Software Development

Consensus Software Development

Bitcoin as a decentralized system,

- it has no “authority”
- to impose its will
- on the participants of the network

Diffusion of power

- Power is diffused
- between multiple constituencies

Consensus Software Development

Power holders

- miners,
- Core developers,
- wallet developers,
- exchanges,
- merchants,
- end users

Common decision

- Decisions cannot be made unilaterally
- by any of these constituencies

Consensus Software Development



there is no perfect solution

for consensus development



change is difficult

consensus forces compromise

one constant characteristic of consensus software development



Weakness or Strength?

Some see this as a weakness of consensus systems.

Some others see it, as the system's greatest strength.

Discussions

Your opinion

What do you think on this chapter?

Challenging

What is the most interesting point?

Clarification

What are the unclarified points?

Critical issue

Select and explain a critical issu in this chapter

